UWE Bristol | University of the West of England

UFCFTR-30-3
Distributed & Enterprise Software Development

Requirements Handling

UWE Bristol University of the West of England

Session Outline

- SWEBOK - SDLC
- Required large scale system behaviour
- Stakeholders
- Functional & Non-Functional Requirements
- User stories / Use cases

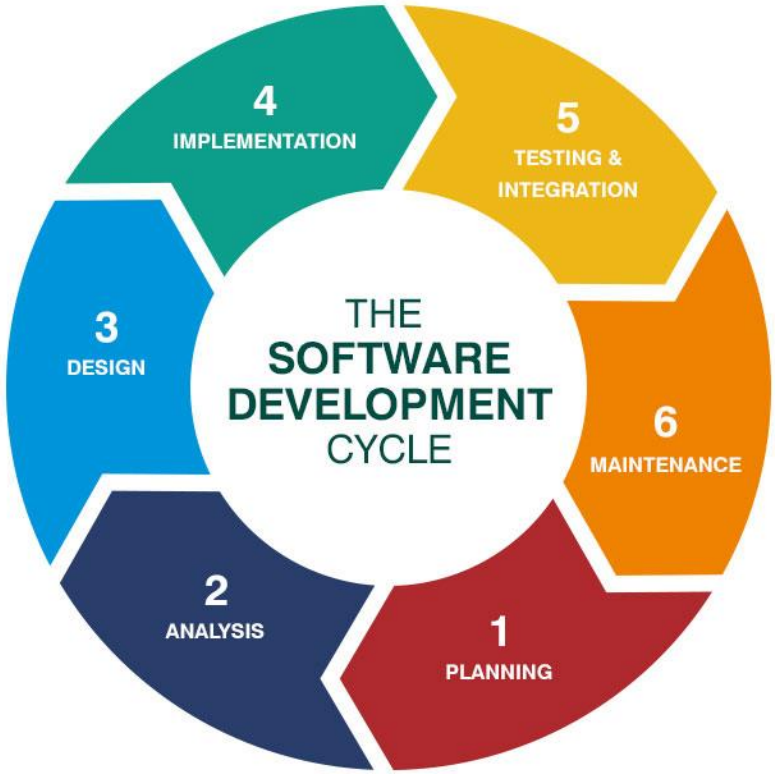
Systems Development Life-Cycle

Application development life-cycle

Or

Software development life-cycle

- Software requirements
- Software design
- Software construction
- Software testing
- Software maintenance
- Software configuration management
- Software engineering management
- Software engineering process
- Software engineering tools and methods
- Software quality

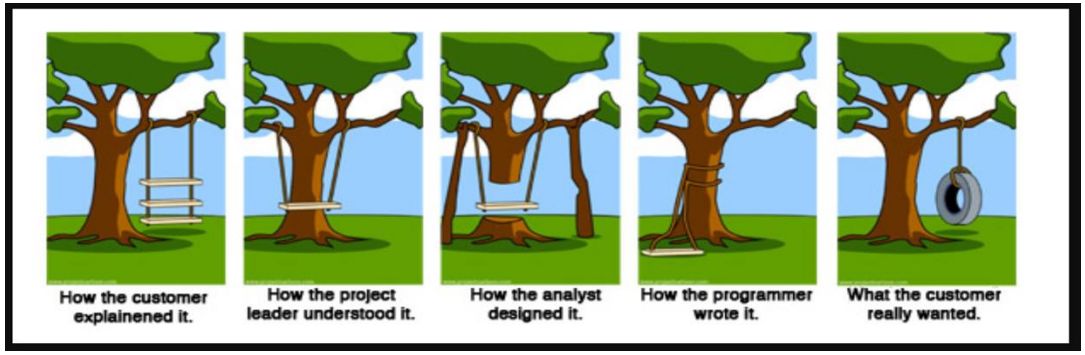


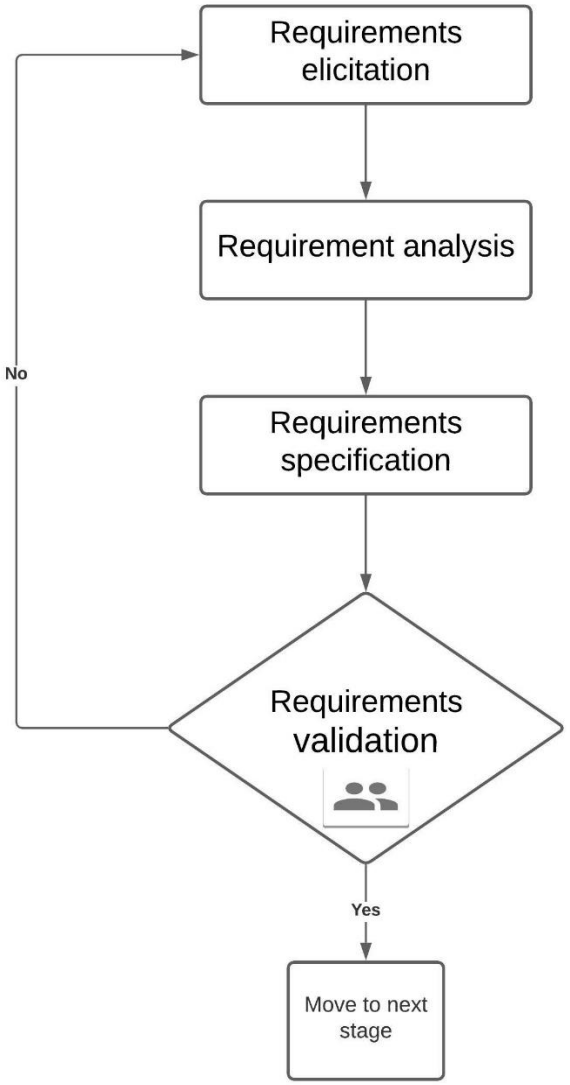
SWEBOK®

Software Requirements

"A requirement is defined as a property of a real-life problem that must be exhibited by the software and the use that is given to it"

- Requirement process
  - Requirements elicitation
  - Requirement analysis
  - Requirements specification
  - Requirements validation
  - Requirements signoff





Software Design

"Software design is the process of defining the characteristics of the system, its component and results"

Key issues in software design:

- Software structure and architecture
- Software design quality analysis and evaluation
- Software design notations
- Software design strategies and methods

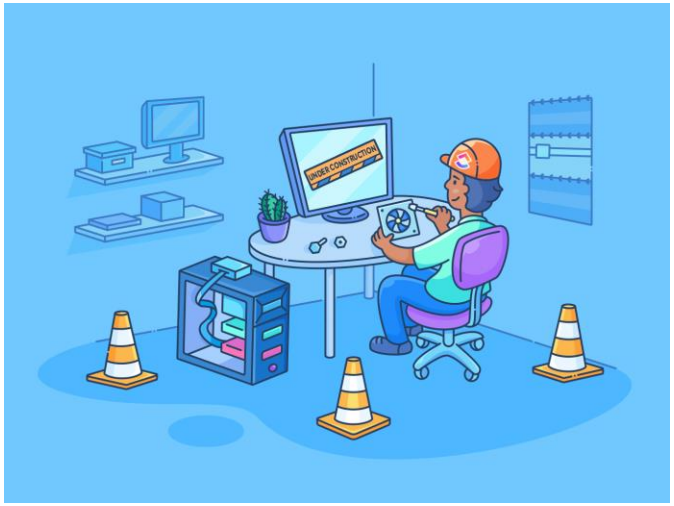


"...And that, in simple terms, is what's wrong with your software design."

Software Construction

- Software construction is defined as the detailed creation of working in meaningful software, going through, and combining, the activities of coding, verification, unit testing, integration testing and debugging

- Managing construction:
  - Minimising the complexity
  - Anticipation of change
  - Construction for verification and the use of standard



Software Testing

- “Testing is the activity of evaluating the product's quality, identifying the problems and defects, commonly referred as bugs, and improving the product by correcting them”

- Test levels
  - Unit testing
  - Integration testing
  - Systems Testing
- Test related measures
  - Test Coverage
    - How much of the application was tested?
  - Test effectiveness
    - How good were the tests?



Software Maintenance

- “The maintenance is defined as the activities required to provide cost effective support to a software product.”
- Modification of software after the delivery Correction of faults
- Improvement of software qualities
- Types of Maintenance
  - Corrective
  - Adaptive
  - Perfective
  - Preventive

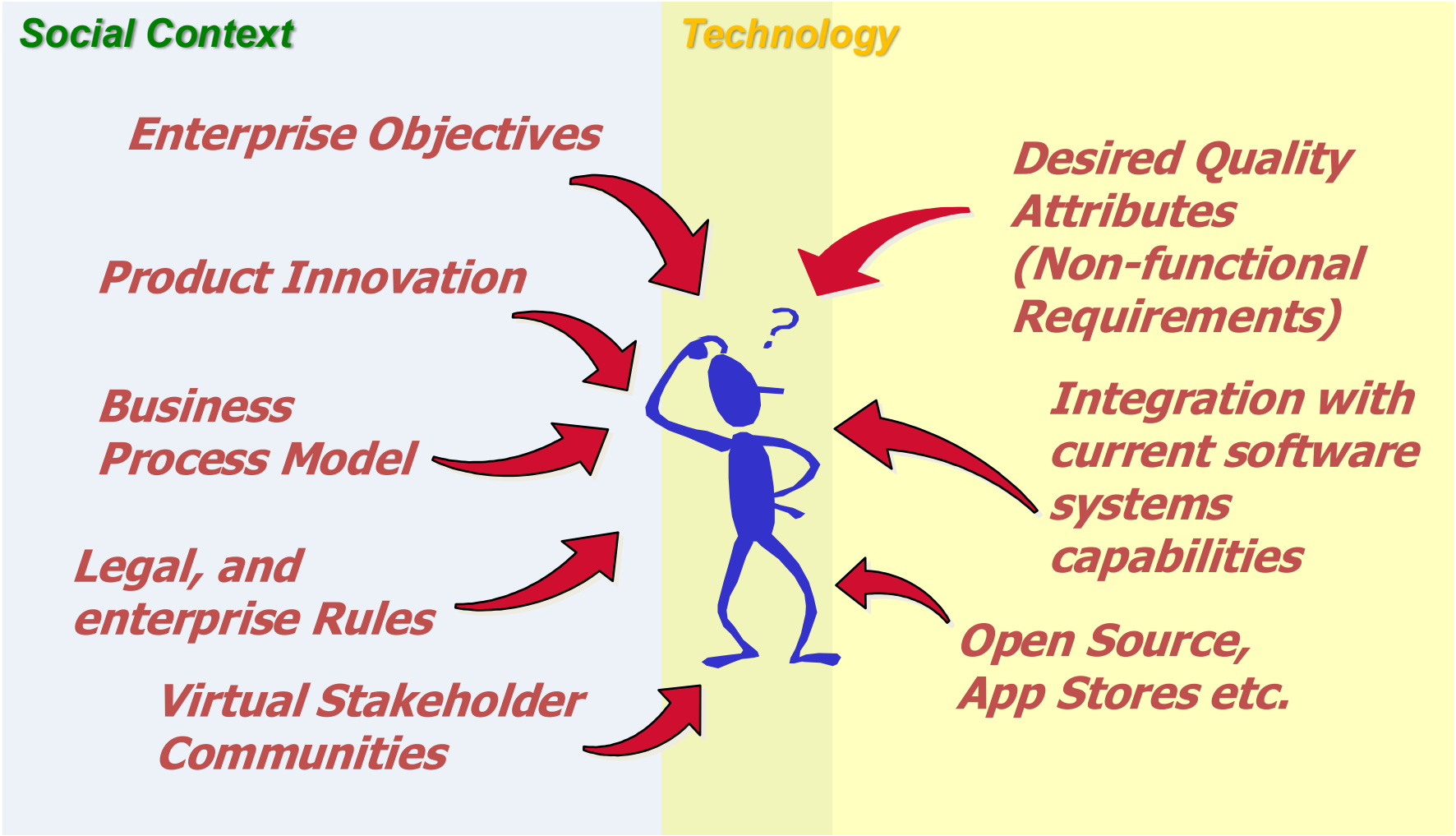


Requirements Capture

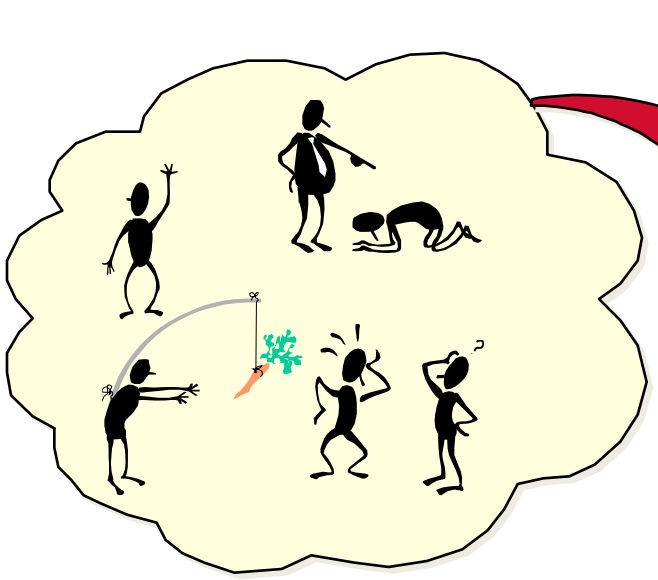
Who are the stakeholders – ***local*** or ***virtual?***

What do they want?

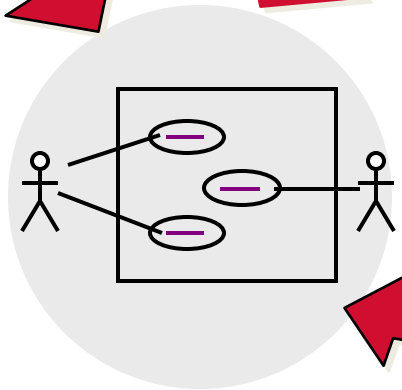
Large Scale Systems: Requirements



Stakeholders



Development Team





Problem Domain Experts, User Groups



Other stakeholders, e.g., financial backers

Communication,
Comprehension,
Completeness, etc.

What are Requirements?

- A specification of what should be implemented
  - a description of system behaviour
  - a system property or attribute
  - a constraint on the development process of a system

- Requirements are not
  - schedule constraints
  - design or implementation details
  - tests
  - but … they must be **testable**



Goals of Requirements

- What are requirements for?
  - creating common understanding between stakeholders
  - defining clear project objectives
  - find out what the user needs
  - find out what the user really needs!
- Common understanding between stakeholders
  - external system behaviour
  - system constraints
  - business rules
  - priorities
- Building the right system
  - determining what gets built!

Functional Requirements

- Capture intended behaviour of system
  - services, tasks or functions system required to perform

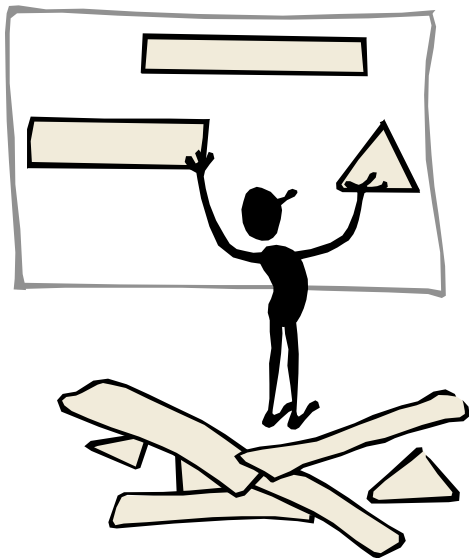
- Functional requirements
  - might be represented as use cases & scenarios, activity diagrams
  - relevant to users, sponsors developers, testers, assessors, and communicators
  - owned by analyst



Non-functional Requirements

- Property or characteristic system must exhibit
    - possibly constraint it must respect
    - not an observable system behaviour

- Sources of non-functional requirement
    - business rules
    - quality attributes
    - external interfaces
    - design and implementation constraints



User Stories

- “In software development and product management, a user story is one or more sentences in the everyday or business language of the end user or user of a system that captures what a user does or needs to do as part of his or her job function.
- User stories… …capture the 'who', 'what' and 'why' of a requirement in a simple, concise way, often limited in detail by what can be hand-written on a small paper notecard”.

http://en.wikipedia.org/wiki/User_story

UWE Bristol University of the West of England

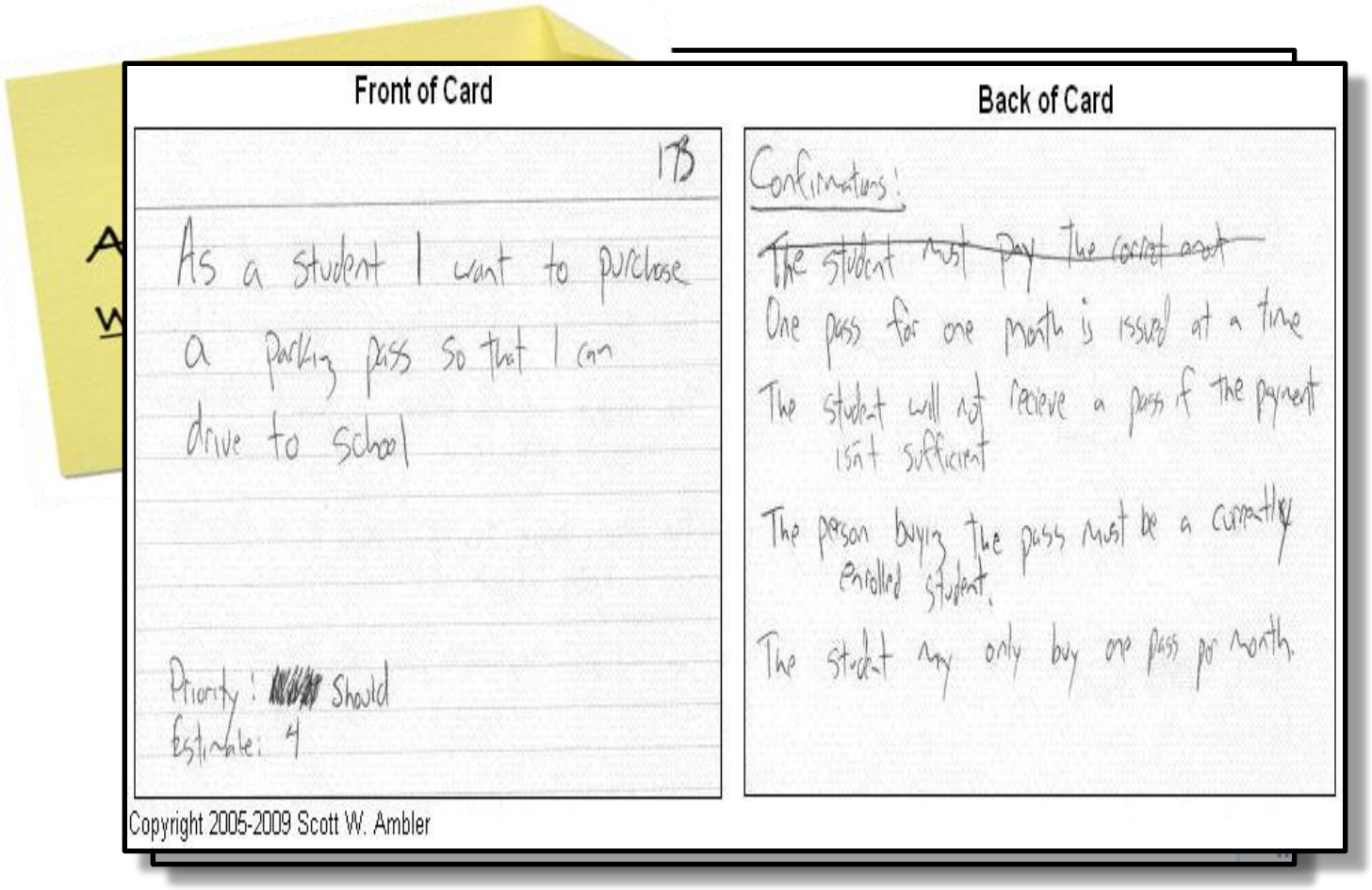
Common User Story Template

As a <role> I can <capability>, so that <receive benefit>

In order to <receive benefit> as a <role>, I can <goal/desire>

As <who> <when> <where>, I want <what> because <why>

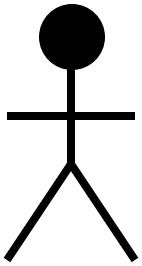
User story detail - acceptance criteria



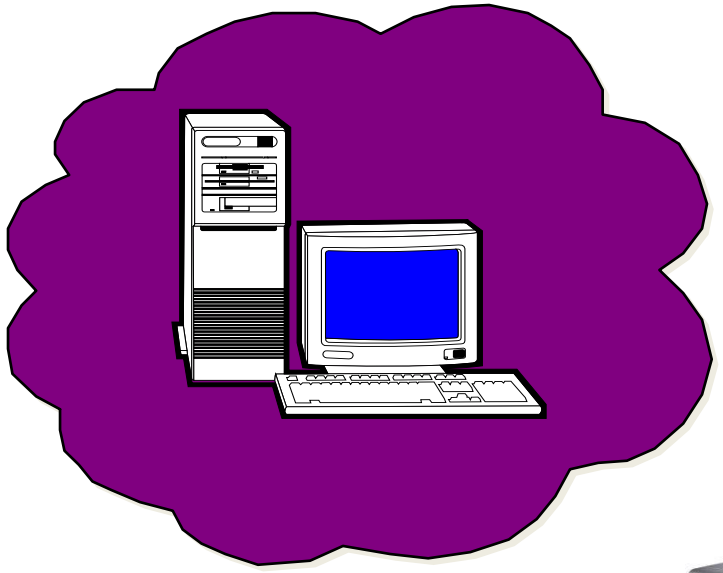
https://www.mountaingoatsoftware.com/blog/preview/1691

Use Cases – What is an Actor?

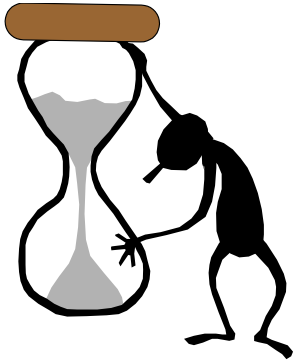
Human being in a Role



E.g., Sales Person



Time-based events



E.g., End of Day

Other external components, or Systems



External to (i.e. NOT part of) the system

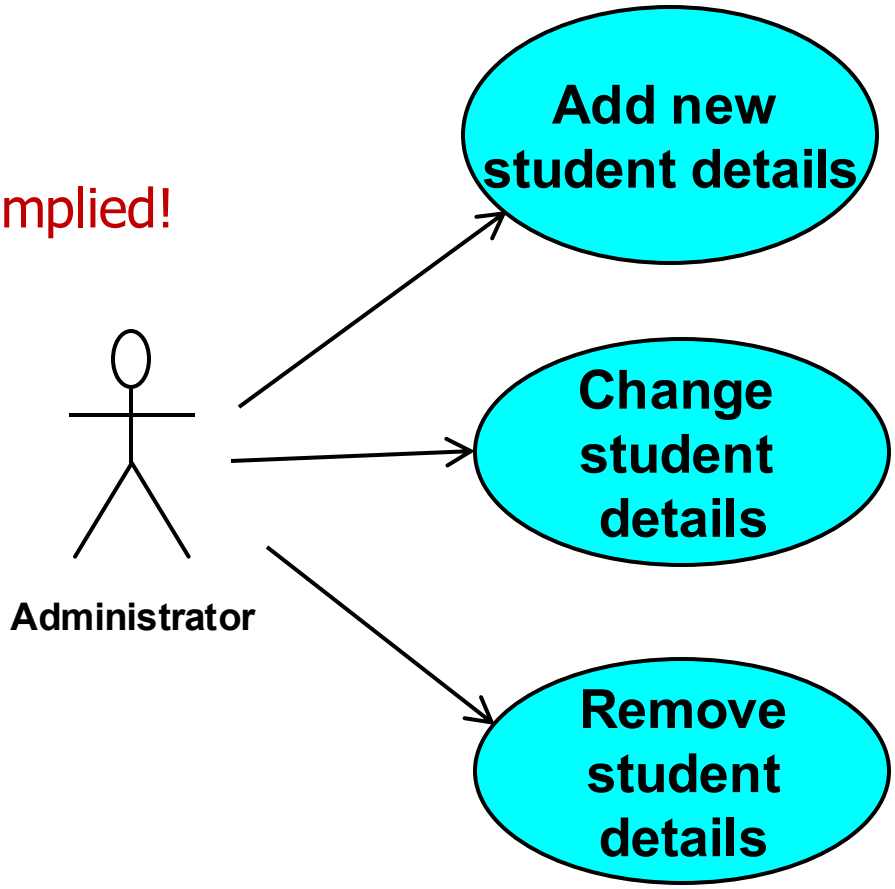
UWE Bristol University of the West of England

Use Cases – What is a Use Case?

1. “Black Box” view of software behaviour
2. Textual narrative (user story)
3. Unit of interaction between actor and system
4. What the system does, not how
5. Sequence / Flow yielding result of value to actor – a goal
6. Sum of all use cases represents the scope of large-scale software system

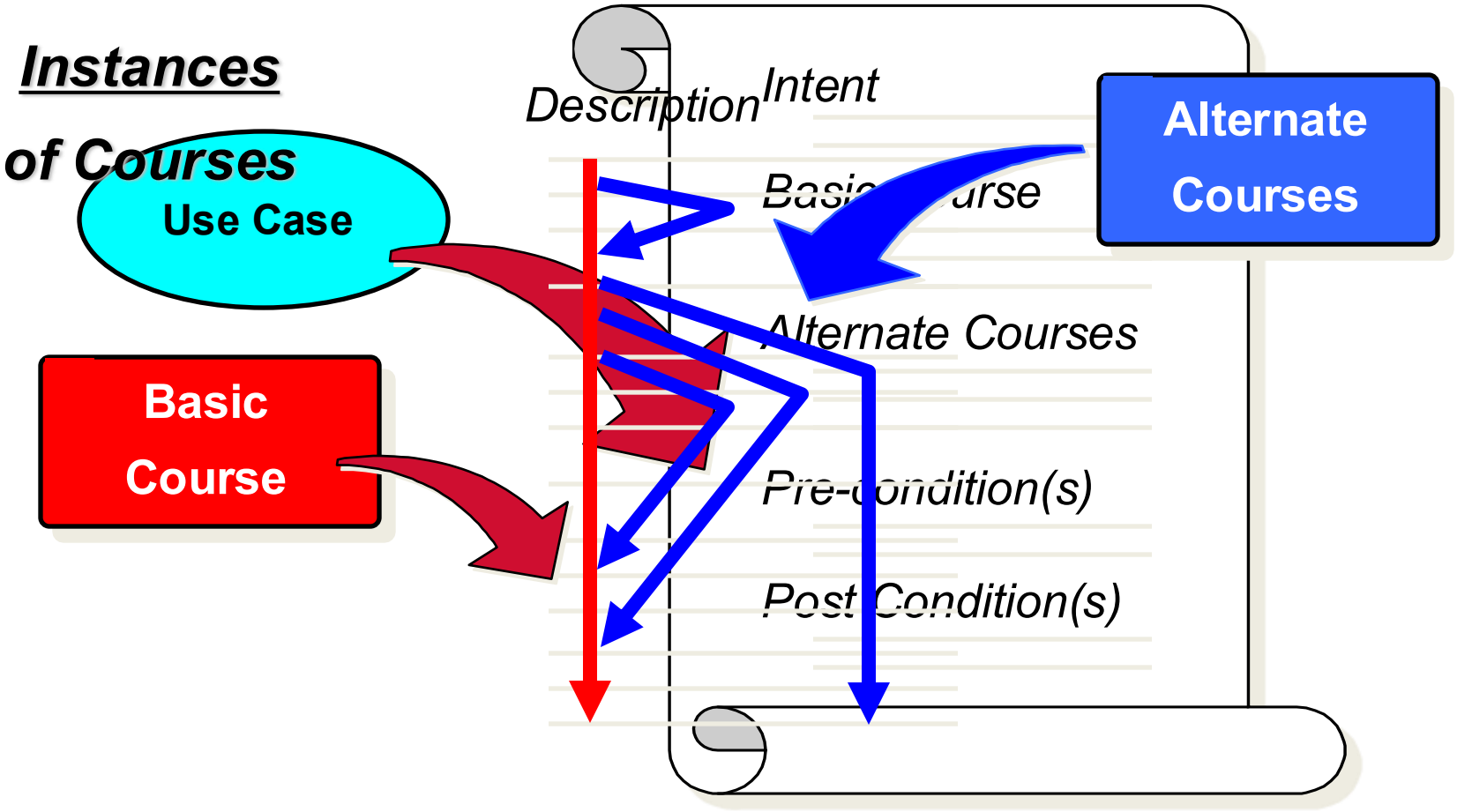
UML Use Case Diagram

Sequence NOT implied!

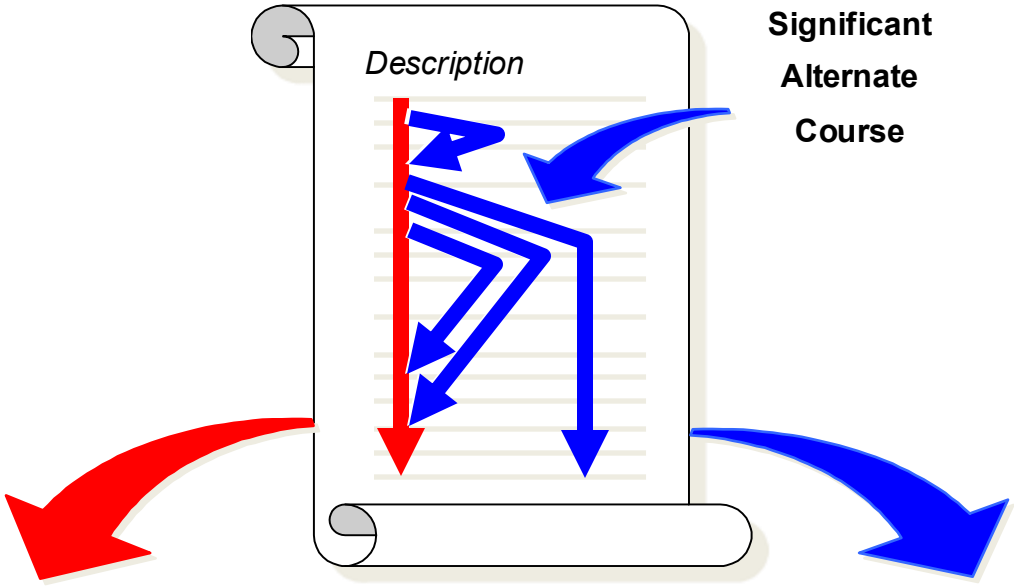


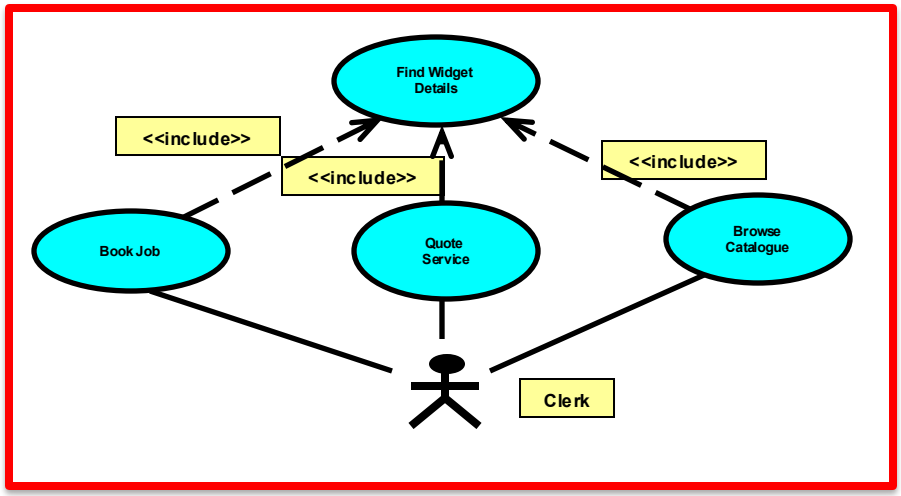
E.g., Student Management System

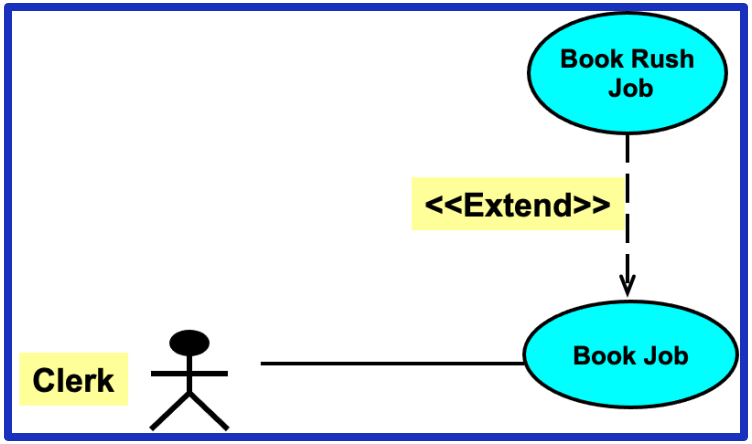
Use Case Definition



Use Case Relationships







UWE Bristol | University of the West of England

Any Questions?

